

TECHNICAL REFERENCE

Badminton Video Analysis Pipeline

Full documentation of every stage: raw video in, winner + win-reason + PDF report out. Inputs, outputs, parameters, and why each part of the system is built the way it is.

`pipeline_release/docs/` · generated reference

Badminton Pipeline Documentation

Contents

Pipeline Documentation — Overview

- Contents
- The two run modes
- Where output lives
- Design threads that run through every stage

Stage 0 & 2 — Video Preprocessing and Court/Net Setup

- Why this exists
- Stage 0 — `video_utils.ensure_preprocessed()`
- Reference-frame selection — `frame_picker.py`
- Manual annotation — `analysis/annotate_court.py`
- Automatic annotation — `analysis/auto_court.py`
- Parameters reference

Stage 2b–2e — Court Presence Detection and Segmentation (`--broadcast` mode only)

- Why this exists
- `court_presence.py` — the per-frame signal
- `extract_segments.py` — turning the signal into clips
- Per-segment processing (`run_full_pipeline.py` , not a separate file)
- `stitch_segments.py` — remapping back to one global timeline
- Parameters reference

Stage 0, 1, 4 — Shuttle Detection, Filtering, Gap-Filling

- Stage 0 — `TrackNetV3/predict.py` : shuttle detection
- Stage 1 — `TrackNetV3/filter_trajectory.py` : removing fake detections
- Stage 4 — `TrackNetV3/fill_gaps.py` : filling real occlusion gaps
- Parameters reference

Stage 3 — Player Detection

- How it works
- Parameters reference
- Why this file specifically (design note)

Stage 5 — Rally Detection

- The core idea
- Pipeline, in order
- The state machine
- `filter_pickup_rallies()` — floor-pickup and static-noise rejection
- `apply_lookback()` — recovering the true start
- `merge_close_rallies()` — residual split repair
- Parameters reference

Stage 6 (part 1) — Win Prediction

- Court geometry setup (`rule_engine.load_court_from_json`)
- Player side assignment (`assign_player_sides`)
- The rule engine (`analyze_rally_end`)
- The CNN ensemble (`predict.py`)
- The trajectory Random Forest (`traj_rf.py`)
- Fusion strategy — which prediction wins
- Known limitation — the stationary-shuttle detector often finds nothing
- Parameters reference

Stage 6 (part 2) — Fusion, Commentary, and PDF Report

- `fusion_layer.py` — merging with the shot classifier
- `run_unified_pipeline.py` — the actual orchestration
- Parameters / config reference

Stage 7 — Verification Video

- Why it exists
- Two purpose-built modes
- Overlay gating — court/net only where the court actually is
- Rendering
- Output codec — the one non-obvious part
- Parameters reference

Web Review App

- `jobs.py` — running the real pipeline as a background job
- `server.py` — the Flask API
- Frontend — `index.html` + `app.js`
- Parameters reference

Pipeline Documentation — Overview

This is the technical reference for the full badminton video analysis pipeline: raw video in, winner + win-reason + PDF report out, with a human review step in between. Each stage has its own doc file; this page explains how they fit together, the two run modes, and where to look for what.

Contents

File	Stage(s)	What it covers
01_video_preprocessing_and_court_setup.md	0, 2	Codec/resolution normalization, manual + automatic court/net annotation, the temporal-stability reference-frame picker
02_court_presence_and_segmentation.md	2b–2e	Broadcast-mode only: detecting which time ranges show the real court, cutting/stitching segments
03_shuttle_tracking.md	0–1, 4	TrackNetV3 shuttle detection, the 3-filter trajectory cleaner, gap-filling
04_player_detection.md	3	YOLO11 player detection + court-boundary filtering
05_rally_detection.md	5	Segmenting the cleaned shuttle trajectory into individual rallies
06_win_prediction.md	6	Rule engine + CNN ensemble + trajectory Random Forest → winner, win-reason, score
07_fusion_commentary_pdf.md	6 (cont.)	Merging with the shot classifier, deterministic stats summary, LLM commentary, PDF report
08_verification_video.md	7	The combined court+net+player+shuttle annotated video, rally-detected markers
09_web_review_app.md	—	The Flask + vanilla-JS app: upload → watch it process → review/correct → save

The two run modes

Everything is orchestrated by `run_full_pipeline.py`. It has two modes, chosen by `--broadcast`:

Default (pre-trimmed clip) — assumes the whole input video is rally footage, no intro/crowd/replay cutaways. Runs shuttle detection and player detection on the *entire* video, once each.

```
preprocess → [0] shuttle detect → [1] filter trajectory → [2] court+net
→ [3] player detect → [4] gap-fill → [5] rally segment → [6] win predict
+ fusion + PDF → [7] verification video
```

--broadcast — for a full match broadcast where the camera cuts away to intro graphics, crowd shots, replays, and coach close-ups between rallies. Running shuttle/player detection on the *whole* video would waste most of the compute on footage that isn't even the match. Instead:

```
preprocess → [2] court+net (wide-search reference frame)
→ [2b] court-presence scan → [2c] extract court-present segments
→ [2d] per-segment: shuttle detect + filter + player detect
→ [2e] stitch segments back into one global timeline
→ [4] gap-fill → [5] rally segment (gated by court-presence too)
→ [6] win predict + fusion + PDF → [7] verification video
```

The step numbers above (0/7, 2b/7, etc.) are exactly what `run_full_pipeline.py` prints to stdout for each stage — useful for matching a log line back to the code that produced it, and it's what the web review app's progress parser watches for (see [09_web_review_app.md](#)).

Every stage checks whether its output file already exists and skips itself if so — a failed or interrupted run can just be re-invoked with the same `--match_name` and it picks up where it left off, without redoing finished work.

Where output lives

Everything for one match lands in a single folder, `<match_name>/`, at the repo root (`pipeline_release/<match_name>/`):

<code><match_name>/</code>	
<code><match_name>.mp4</code>	preprocessed input video
<code><match_name>_court.json</code>	court corners + net position
<code><match_name>_court-presence.csv</code>	per-frame court-visible signal (broadcast mode only)
<code><match_name>_ball_filled.csv</code>	final shuttle trajectory (gap-filled)
<code>player_detections.csv</code>	player positions per frame
<code><match_name>_rally.csv</code>	rally start/end frames
<code>win_predictions.csv</code>	per-rally winner/reason/score (the core output)
<code><match_name>_annotated.mp4</code>	verification video
<code>segments/</code>	broadcast mode only: per-segment clips +
<code>intermediate CSVs</code>	
<code>unified/</code>	
<code>merged_analysis.csv</code>	win_predictions + shot classifier merged
<code>badminton_analysis_enhanced.csv</code>	fixed 8-column schema for the BadmintonAnalysis viewer
<code>analysis_report.txt</code>	deterministic stats + AI commentary
<code><players>_badminton_highlights_<timestamp>.pdf</code>	
<code>corrections.json</code>	human review edits (web app only; never overwrites the above)

Design threads that run through every stage

A few decisions repeat across multiple files, worth knowing up front rather than re-explaining in each doc:

- **Homography, not raw pixels, for anything "real-world."** Court corners define an image → metres transform (`cv2.findHomography` / `cv2.getPerspectiveTransform`, singles court = 5.18m × 13.4m). Boundary checks, the trajectory RF's `margin_cm` feature, and the court-presence geometric check all go through this rather than comparing raw pixel coordinates, because pixel distance means something different depending on where on the court you are (perspective).
- **A homography only correctly maps points on the plane it was built from.** The court corners are on the ground; the shuttle is airborne for almost its whole trajectory. Transforming a mid-flight pixel position through the ground homography gives a real but *wrong* answer — where a ground-level point at that pixel would be, not where the actual 3D shuttle is. Every place this matters (fault-side detection, the RF's landing features) deliberately uses positions near actual ground contact, not arbitrary in-flight frames.
- **Cheap gate before expensive check, everywhere there's a two-stage filter.** Court presence checks green-fraction before running homography verification; win prediction checks CNN confidence before running the RF; gap-filling checks a scene mask before computing interpolation. The pattern shows up because it's usually 10-100x cheaper to reject on the first signal than to always run the expensive one.
- **Self-consistency over absolute precision when the reference is uncertain.** The court-presence homography check calibrates against the *same* detector's output on the reference frame, not hand-clicked ground-truth corners — a raw contour-blob detector and a human click disagree systematically even on the identical frame, so comparing detector-to-detector (not detector-to-click) is what actually measures "did the camera move," not "how sloppy is the detector."
- **Everything that touches a video file re-validates it first.** `video_utils.ensure_preprocessed()` — codec, resolution, fps — because `cv2.VideoCapture` silently fails to decode some real-world inputs (AV1 in particular) with no exception, just `.read()` returning `False` forever, which otherwise surfaces as a confusing unrelated error far from the real cause.

Stage 0 & 2 — Video Preprocessing and Court/Net Setup

Files: `analysis/video_utils.py`, `analysis/annotate_court.py`, `analysis/auto_court.py`, `analysis/frame_picker.py`, `analysis/extract_features.py` (corner/net functions only — the rest of that file is covered where it's actually used, in [06_win_prediction.md](#)).

Why this exists

Everything downstream — court-presence detection, player filtering, the fault-side rule engine, the trajectory RF's `margin_cm` feature — depends on one thing being right: 4 pixel coordinates for the court corners, plus where the net is. Get this wrong and every later stage is wrong in a way that's hard to notice, because the pipeline will still *run* and produce plausible-looking numbers. So this stage gets disproportionate engineering attention relative to how simple it sounds ("click 4 points").

Stage 0 — `video_utils.ensure_preprocessed()`

Input: any video file. **Output:** a path to a video that's guaranteed H.264, 1280×720, 30fps.

Why: `cv2.VideoCapture` cannot reliably decode AV1 (common for YouTube downloads — yt-dlp often defaults to it). It doesn't raise an exception — `.read()` just returns `False` on every single frame. A script scanning for something (a good annotation frame, court presence) then silently finds nothing and fails with a confusing error far from the real cause ("no frame with a centered court view found" when the actual problem is "this video never decoded a single frame"). Wrong resolution or frame rate is worse — it doesn't fail at all, it silently breaks every pixel-coordinate and frame-count assumption downstream (a `court.json` clicked at 1280×720 applied to a 1920×1080 video, rally duration math off by whatever the fps ratio is).

How it works:

1. `probe_video(path)` — one `ffprobe` call, reads `codec_name`, `width`, `height`, `r_frame_rate`.
2. `_matches(path, w, h, fps)` — `codec == "h264"` and `width==w` and `height==h` and `abs(fps_diff) < 0.1`.
3. If it already matches: return the original path, untouched, zero cost.
4. If not: check for a cached transcode next to it (`<stem>_prepared.mp4`) — reused across repeated calls instead of re-transcoding every time a script touches the same video.
5. Otherwise transcode:
`ffmpeg -vf scale=1280:720 -r 30 -c:v libx264 -preset fast -crf 23 -an`. Audio is dropped (`-an`) — nothing downstream ever reads it.

Where it's called: `run_full_pipeline.py` calls it once, up front, before any stage runs. But `annotate_court.py`, `auto_court.py`, and `court_presence.py` **also** call it themselves (each has a `--no_preprocess` opt-out flag) — because those three scripts are also meant to be runnable standalone, not just through the orchestrator, and standalone runs don't get the orchestrator's up-front normalization for free.

Reference-frame selection — `frame_picker.py`

Both annotation paths (manual and automatic) need to pick **which single frame** to show/detect on. This module is shared between them so they agree on what "a good frame" means.

The problem with a fixed frame number

A fixed early frame (e.g. frame 90, ~3s in) is fine for a pre-trimmed rally-only clip, but wrong for a full broadcast — verified on a real 31-minute match: frame 90 was a drone shot of the arena exterior, not the court.

`center_score()` / `margin_score()`

Two pure scoring functions on a candidate frame's detected court corners:

- `center_score`: 1.0 if the court's centroid is dead-center in frame, 0 at the edge.
- `margin_score`: 1.0 if there's clear breathing room on all sides (the standard wide broadcast shot); drops toward 0 as the court approaches touching a frame edge (a zoomed/cropped shot).

Combined as `score = center * margin * min(1, area_frac / 0.30)` — area is a **gate** (`min_area_frac`, default 0.15 of frame area — must be a substantial, plausible court view) not the ranking signal; center/margin decide between candidates that all clear that bar. This deliberately rewards the standard centered wide shot over a bigger but zoomed/cropped one.

`pick_frame_fast()` — the actual search used in practice

Full-video scanning (`rank_candidate_frames`, below) can be hundreds of frame reads on a long broadcast — minutes of wall time. `pick_frame_fast` is the practical default:

1. Try **one** candidate first: the video midpoint (a strong prior — broadcasts are front/back-loaded with intro, walk-ins, post-match ceremony, so the middle is very likely mid-match play), or `start_frame` if the caller already knows roughly where play starts.
2. If that frame's score clears `GOOD_ENOUGH_SCORE = 0.5`, return immediately — one frame read total.
3. Otherwise step forward `step_sec` (default 5.0s) at a time, up to `max_fallback_frames` (default 20) more candidates, and return the best scored one found.

Bounded cost: at most `1 + max_fallback_frames` frame reads, vs. up to hundreds for the full scan. Measured on a real 31-minute broadcast: ~8 seconds instead of 1-2 minutes.

`_stable_enough()` — the temporal-stability gate

A frame can score well on area/center/margin while still being a *bad* reference — e.g. one well-composed frame mid-camera-pan. Since every later frame in the video gets checked against this one reference via homography (see [02_court_presence_and_segmentation.md](#)), an unrepresentative reference silently causes the real match footage to fail a check it should pass.

How it works: given a candidate frame's own detected corners, build a homography from *those* corners (self-referential — not the eventual manually-clicked or line-refined corners, which don't exist yet at this point), then sample a window (`window_sec=8.0` , `sample_stride_sec=1.0`) of frames around the candidate. For each sample: green-fraction gate (`green_threshold=0.55`) then homography reprojection-error check (`max_reproj_error_m=0.6`). Find the longest contiguous run of "passing" samples that includes the candidate itself, convert to seconds. Require `>= min_present_sec = 3.0` seconds of genuine surrounding consistency before accepting the candidate at all.

This is wired as a **hard gate**, not just an informational score — inside `_score_one()` , a candidate that fails stability is treated identically to "no court detected," and the search moves to the next candidate. Verified on real footage: the video's midpoint frame (previously always accepted) correctly got rejected once for only 1.0s of surrounding consistency, and the search kept going until it found a genuinely stable one.

Escape hatch: `--no_stability_check` on both `annotate_court.py` and `auto_court.py` skips this and falls back to the single-frame score alone — faster, but risks exactly the failure mode above.

`rank_candidate_frames()` — the exhaustive alternative

Same scoring, but scans the **whole** video at `step_sec` intervals (default 400 candidates max) and returns every one that clears the area gate, sorted best-first. Used when `pick_frame_fast` can't find anything good nearby (`--full_scan` flag), or to populate the "Try Different Frame" button's full alternative list in the manual annotation UI.

Manual annotation — `analysis/annotate_court.py`

A self-contained browser tool: no dependency on the rest of the codebase's web stack, just Python's `http.server` .

Flow:

1. Pick the frame to show (see above): `--frame N` (exact), `--auto_frame` (search via `frame_picker`), or default frame 90.
2. Serve one HTML page with the frame embedded as a base64 JPEG. Click 8 points in order — `TL`, `TR`, `BR`, `BL` (court corners), `NL`, `NR` (net post bases — ground-level, gives `net_Y`), `CL`, `CR` (net cable top — gives `net_top_Y`).
3. Each click POSTs to `/click` ; "Save" POSTs the full 8-point list to `/save` , which writes the JSON and shuts the server down.

Output schema (`<match>_court.json`):

```
{
  "frame_no": 8634,
  "frame_size": {"width": 1280, "height": 720},
  "corners": {"TL": [...], "TR": [...], "BR": [...], "BL": [...]},
  "net": {
    "net_Y": ..., "net_top_Y": ...,
    "left_bottom": [...], "right_bottom": [...],
    "cable_left": [...], "cable_right": [...]
  },
  "source": "manual"
}
```

`frame_no` matters beyond record-keeping — `court_presence.py`'s self-calibration re-reads exactly this frame (see next doc) to remove a systematic detector-precision bias.

Why this specific implementation (`ThreadingHTTPServer` + `lock` + `per-request timeout`), not the obvious `http.server.HTTPServer` : the plain single-threaded server handles one request at a time on the main thread. If a client's connection stalls mid-response (flaky network, a browser tab that stops reading), `wfile.write()` can block indefinitely with **no timeout**, freezing the entire server for every subsequent request — including Save, from the *same* browser tab. This is not a hypothetical: it hung a real session for 5+ minutes, process alive, socket still accepting connections, never responding.

`ThreadingHTTPServer` puts each request on its own thread; `Handler.timeout = 30` bounds how long any one of them can block; `STATE_LOCK` serializes the shared-state mutations (`points` , `bgr` , `args.frame` , `candidate_idx`) since multiple request threads can now genuinely run concurrently.

Automatic annotation — `analysis/auto_court.py`

Zero-click alternative. Two-stage corner detection:

Stage 1 — green-mask blob (`extract_features.try_detect_corners`): HSV-threshold for the court's green surface, adaptive morphological close+open (kernel scaled to image size), take the largest contour, try `cv2.approxPolyDP` at progressively looser epsilons until it collapses to 4 points, sort into `[TL, TR, BR, BL]` order by Y then X (`_sort_corners`). Fast, but this typically extends *past* the painted sideline (sponsor mats, warm-up strip) — it's the extent of the green surface, not the boundary line.

Stage 2 — white-line refinement (`refine_corners_court_type`): detect long white line segments inside that green region (`cv2.HoughLinesP` , `minLineLength = 20% of min(h,w)`), classify each as horizontal (baseline, angle < 25° or > 155°) or diagonal (sideline), cluster near-duplicate detections of the same physical line (`_cluster_1d` , gap threshold 15px).

The genuinely hard part: **a badminton court paints both the doubles sideline and, 0.46m inside it, the singles sideline**. "The outermost white line" is not automatically the right one for a singles match. The code checks the separation between the two clusters on each side (`SEP_MIN_PX=15` , `SEP_MAX_PX=150`) — the physically-plausible range for a 0.46m gap at this camera's typical scale) and

only proceeds if **both** sides show a confidently-separable inner/outer pair. If it can't tell — only one line detected, or the separation is outside the plausible range — it does **not** guess. It falls back to the green-mask corners with `court_type_matched: false` in the output and a printed warning telling you to use manual mode instead. This was a deliberate fix for a real failure mode: silently returning doubles corners mislabeled as singles.

Net position, two very different derivations:

- `net_Y` (ground line): geometrically exact. `compute_net()` builds a homography from the 4 detected corners to the real-world court rectangle (5.18×13.4m singles / 6.1×13.4m doubles), projects the known net depth (6.7m from baseline) back into image space.
- `net_top_Y` (the elevated cable, ~1.55m up): **cannot** be derived from a flat ground-plane homography at all — that needs the camera's full 3D pose, which isn't being solved for. It's approximated as the mean Y of the TL/TR corners, based on an empirical observation (from manually-annotated production matches) that the cable sits within a few pixels of the far baseline in this camera convention. This is a real approximation, not a measurement — `--net_top_y` lets you override it, and manual annotation (which clicks the actual cable) is more accurate.
- `snap_net()` refines `net_Y` further by detecting the blue net-post stands' pixels in HSV (they sit at exactly the net's depth on both sidelines) within a vertical band around the computed `net_Y`; falls back to `cv2.Canny` + `HoughLinesP` strongest nearby horizontal edge if the blue-post detection finds nothing.

Reference-frame picking for `--wide_search` reuses `pick_frame_fast` and the stability check above — this is the same picker `annotate_court.py --auto_frame` uses, so both annotation paths agree.

Parameters reference

Parameter	Default	File	Meaning
width, height, fps	1280, 720, 30.0	video_utils	Target normalization
GOOD_ENOUGH_SCORE	0.5	frame_picker	Score threshold to accept a candidate without further search
min_area_frac	0.15	frame_picker	Court must cover $\geq 15\%$ of frame area to be a plausible candidate
window_sec	8.0	frame_picker (_stable_enough)	$\pm$ window checked around a candidate for temporal stability
sample_stride_sec	1.0	frame_picker	Sampling interval within the stability window
min_present_sec	3.0	frame_picker	Minimum contiguous stable duration required
max_reproj_error_m	0.6	frame_picker	Max homography reprojection error (metres) counted as "same position"
SEP_MIN_PX, SEP_MAX_PX	15, 150	auto_court	Plausible pixel separation range for doubles ↔ singles sideline gap
EDGE_MARGIN	20px	auto_court, extract_features	Excludes lines right at the green-paint edge (paint boundary, not court line)
shift_limit	30%/15% of frame height (singles/ doubles)	auto_court	Max allowed corner shift from line-refinement before rejecting it

Stage 2b–2e — Court Presence Detection and Segmentation (`--broadcast` mode only)

Files: `analysis/court_presence.py`, `analysis/extract_segments.py`, `analysis/stitch_segments.py`.

Skip this whole stage if you're not running `--broadcast` — a pre-trimmed rally-only clip goes straight from court annotation to shuttle/player detection on the whole video.

Why this exists

A full broadcast isn't one continuous rally-camera shot. It's intro graphics, player walk-ins, crowd cutaways, coach shots, and replays, interleaved with the actual constant-angle rally footage. Running TrackNetV3 and YOLO player detection on all of it wastes most of the compute on footage that isn't even the match. This stage answers one question per frame — "is the main broadcast camera angle on screen right now?" — then uses the answer to cut out just the relevant time ranges.

`court_presence.py` — the per-frame signal

Input: the video + the reference `court.json` from stage 2. **Output:** `<match>_court_presence.csv` — one row per frame: `frame_no, green_frac, reproj_error_m, court_present`.

Two-stage check, cheap first

Stage 1 — green-fraction gate. What fraction of the pixels *inside the reference court polygon* are green (HSV range `[30, 35, 35]` — `[90, 255, 255]`)? One masked pixel count. Rejects most non-court frames (crowd, closeups, graphics) for free. Threshold `0.55`, not near 1.0, because players/shuttle/lines legitimately cover part of the polygon even during real rally play.

Not specific enough alone — verified against real segment output: a frame on a *different* but still-green shot (zoomed in, a different camera angle that happens to fill the same screen region with green) passes this even though the court is not actually in the same place there.

Stage 2 — homography verification, only for frames that pass stage 1 (so the expensive part runs on a minority of candidates). Detect this frame's own court corners with the same raw green-blob detector `auto_court.py` uses, reproject them through a homography, and measure the mean distance (in real-world metres) from where they land to where they should. Small error = genuinely the same court in the same position; large error = different framing.

The self-calibration fix (why this isn't naive)

The homography needs a reference. The obvious choice — the precisely clicked/line-refined corners in `court.json` — is **wrong** for this purpose: comparing a raw green-blob-detected candidate against precisely clicked reference corners produces a large *systematic* error even between the reference frame and **itself** (measured: ~1.5m, well above any sensible tolerance), because the raw blob detector is a looser, wider box than a precise click, not because the camera moved.

The fix (`self_calibrated_reference()`): re-read the *exact* reference frame from the video (using `frame_no`, saved in `court.json` by both `annotate_court.py` and `auto_court.py`), run the **same raw detector** on it, and build the homography from *that* blob instead of the precise corners. Now both sides of every comparison use the same detector, and the systematic bias cancels out. Verified: the same reference frame checked against itself now measures ~0m error (down from ~1.5m); genuine rally frames nearby measure ~0.02–0.6m; a temporally-unstable candidate frame (mid camera-pan) correctly measures much higher.

Falls back to the uncalibrated (precise-corner) homography, with a printed warning, if `frame_no` is missing from the JSON or that frame can't be read — matches the older, less-accurate behavior rather than failing outright.

Sampling: every 30th frame, not every frame

`--stride 30` by default (~1 sample/second at 30fps) — checking every single frame is far more precision than the presence signal needs, and `extract_segments.py`'s padding (1.5s each side) already absorbs the resulting fill lag at segment boundaries.

Implementation detail that matters for speed: the scan does **not** `cap.set()`-seek to each sampled frame. For a long-GOP H.264 file, seeking to an arbitrary non-keyframe position forces the decoder to decode forward from the last keyframe anyway — measured close to zero speedup going from stride=1 to stride=30 with `.set()`. Instead it calls `cap.grab()` (decode only, no BGR conversion — cheap) through every frame, and only `cap.retrieve()` (the actual conversion) on the one frame per stride that's checked. Verified: 6.6x faster (28.6s vs 3m9s on an 18-minute broadcast) with byte-identical output to a full stride=1 scan.

Gaps between samples are filled forward (`pd.merge_asof(..., direction= "backward")`) to produce one row per frame in the output CSV regardless of stride — downstream consumers never need to know the sampling rate.

Final decision + smoothing

```
court_present = 1 iff green_frac >= threshold
                  AND reproj_error_m is not NaN (i.e. it passed stage 1)
                  AND reproj_error_m <= max_reproj_error_m
```

Then a centered rolling **median** filter (`smooth_window=5`) removes single-frame flicker — a player fully covering the polygon for a moment, a smash's motion blur, one bad corner detection — from flipping the signal.

`extract_segments.py` — turning the signal into clips

Input: the presence CSV + the video. **Output:** `segments/seg_NNN.mp4` per segment + `manifest.csv` (`seg_idx`, `start_frame`, `end_frame`, `n_frames`, `clip_path`).

1. **Find contiguous runs** of `court_present == 1`.
2. **Merge runs separated by a short gap** (`min_gap_sec=2.0`) — a brief cutaway or graphic mid-rally shouldn't split one rally's footage into two segments.
3. **Pad each merged run** (`pad_sec=1.5`, clamped to valid frame range) — rally starts/ends often dip near the presence threshold as players cross the polygon boundary; re-merge afterward in case padding caused adjacent segments to overlap.
4. **Drop anything still too short** (`min_duration_sec=3.0`) — not a real rally-length span.
5. **Cut each survivor out with ffmpeg's `select` filter**, not `-ss/-to` timestamp seeking — frame-accurate, so segment N's local frame 0 is *exactly* global frame `start_frame`, which the stitching step depends on for correctness. `-c:v libx264 -preset veryfast -crf 18`, no audio.

Errors loudly (`sys.exit`) if zero segments survive — pointing at the two most likely causes (presence threshold, or the court annotation not actually matching this video's camera angle) rather than silently producing an empty pipeline run.

Per-segment processing (`run_full_pipeline.py`, not a separate file)

For each `seg_NNN.mp4` in the manifest: TrackNetV3 shuttle detection → trajectory filtering → YOLO player detection — the exact same stages as [03_shuttle_tracking.md](#) and [04_player_detection.md](#), just run on the short clip instead of the whole broadcast. Segment output filenames are renamed to the `seg_NNN` pattern `stitch_segments.py` matches on (TrackNetV3 names its output after the input clip file, not the segment tag).

`stitch_segments.py` — remapping back to one global timeline

Input: the manifest + a glob pattern matching each segment's local output CSV. **Output:** one CSV with frame numbers remapped to the original video's global numbering, concatenated.

For each matched file: extract its segment index, look up that segment's `start_frame` in the manifest, add it to every local frame number in the file (`df[frame_col] += start_frame`), concatenate.

Extracting the segment index is the one genuinely tricky part of this file, and it was wrong twice before landing on the current implementation:

- **First bug:** searching for `seg_(\d+)` in the *first match found in the full path* — if the match folder itself happens to be named something like `seg_009` (e.g. from an uploaded file called `seg_009.mp4`), every file's path starts with `.../seg_009/...`, which matches *before* the real `seg_000` ever appears in the path. Silently stitches under the wrong segment index.

- **Second bug (the "fix" that broke a different case):** restricting the search to the filename only. Fixes the match-folder-name collision, but breaks player-detection CSVs specifically — their segment index lives in the **parent directory** name (`segments/players/seg_000/player_detections.csv`), not the filename (`player_detections.csv` , fixed, no index in it at all). Shuttle CSVs (`seg_000_ball_clean.csv`) have it in the filename; player CSVs don't.
- **Actual fix:** take the **last** `seg_NNN` match anywhere in the full path (`list(seg_re.finditer(path))[-1]`), not the first, and not filename-restricted. The match-folder name — whichever form it takes — always appears *earlier* in the path than the real segment reference, so "last match" is correct for both the filename-encoded and directory-encoded layouts simultaneously. Verified directly against both real shuttle and player CSVs from the same failing match folder.

`run_full_pipeline.py` calls this twice — once for shuttle (`--frame_col Frame`), once for player detections (`--frame_col frame_no`) — writing into the same paths the non-broadcast mode would have produced directly, so every stage after this point (gap-filling, rally detection, win prediction) doesn't need to know or care whether `--broadcast` was used.

Parameters reference

Parameter	Default	File	Meaning
<code>threshold</code>	0.55	<code>court_presence</code>	Stage-1 green-fraction gate
<code>max_reproj_error_m</code>	0.6	<code>court_presence</code>	Stage-2 homography error gate (metres)
<code>stride</code>	30	<code>court_presence</code>	Frames between samples (~1/sec at 30fps)
<code>smooth_window</code>	5	<code>court_presence</code>	Median-filter window to remove flicker
<code>min_gap_sec</code>	2.0	<code>extract_segments</code>	Merge runs separated by less than this
<code>pad_sec</code>	1.5	<code>extract_segments</code>	Padding added to each side of a segment
<code>min_duration_sec</code>	3.0	<code>extract_segments</code>	Drop segments shorter than this after padding

Stage 0, 1, 4 — Shuttle Detection, Filtering, Gap-Filling

Files: `TrackNetV3/predict.py`, `TrackNetV3/filter_trajectory.py`, `TrackNetV3/fill_gaps.py`.

Three-stage pipeline, each stage cleaning up what the previous one produced, applied in this order every time:

```
raw video → predict.py → _ball.csv (raw detections + noise)
           → filter_trajectory.py → _ball_clean.csv (fake detections removed)
           → fill_gaps.py → _ball_filled.csv (real occlusion gaps interpolated)
```

Stage 0 — `TrackNetV3/predict.py`: shuttle detection

Input: a video file + a TrackNet checkpoint (`ckpts/TrackNet_best.pt`). **Output:** `<name>_ball.csv` — `Frame, X, Y, Visibility, Confidence` per frame.

This wraps a pretrained TrackNet model (heatmap-based shuttle detector, architecture and training defined outside this pipeline — loaded here as a checkpoint dependency, not re-derived). What this file owns:

Sliding-window streaming inference. `VideoIterableDataset` reads the video in overlapping sequences (`seq_len` frames per window, `sliding_step=1`) rather than loading the whole video into memory — necessary for full-length broadcasts. Because windows overlap, **the same frame gets predicted multiple times** (once per window it appears in).

Heatmap → position. For each frame's predicted heatmap: binarize at `0.5`, and only accept a detection at all if the raw (pre-binarize) max value clears `CONF_TH = 0.25`. If it does, `predict_location()` (from `test.py`, not re-derived here) extracts a bounding box from the binary heatmap; the shuttle position is that box's center, rescaled from the model's fixed input resolution (`WIDTH × HEIGHT`, from `utils.general`) back to the source video's actual resolution via `img_scaler = (w/WIDTH, h/HEIGHT)`. Below the confidence threshold: `(0, 0, Visibility=0)`.

Deduplication. Because of the sliding-window overlap, multiple rows can exist for the same `Frame`. Resolved by sorting `[Frame, Visibility desc, Confidence desc]` and keeping the first row per frame — i.e., always prefer a visible detection over an invisible one for the same frame, and among visible ones, the highest-confidence prediction.

Stage 1 — `TrackNetV3/filter_trajectory.py`: removing fake detections

Input: `_ball.csv`. **Output:** `_ball_clean.csv`, same schema, some rows zeroed (`X=0, Y=0, Visibility=0`).

The raw detector produces real position spikes and misidentified objects (racket heads, player limbs, court-line intersections) that look like plausible single-frame detections but aren't physically consistent with shuttle motion. Three filters, applied in sequence, each catching a different failure shape:

Filter 1 — bidirectional spike (`filter_bidirectional_spike`)

For every visible frame, compute speed to the nearest visible neighbor before and after it (using an *original*-value snapshot, so zeroing one frame doesn't cascade and inflate its neighbors' apparent speed). Two independent triggers zero the frame:

- **Hard threshold:** backward speed `> 300 px/frame` with a direct (`dt==1`) neighbor — a single-frame 300+px teleport is physically impossible for *any* shuttle regardless of what follows it.
- **Bilateral threshold:** *both* backward and forward speed `> 130 px/frame`, both direct neighbors — a single-frame spike sitting inside an otherwise-real trajectory.

Filter 2 — internal cluster break (`filter_internal_break`, *iterative*)

Within a run of consecutive visible frames, scan consecutive pairs for a jump `> 300 px/frame` — physically impossible within one continuous detection cluster. At the break, the segment *before* the jump is the fake part if the speed entering that segment was also high (`> 100 px/frame`, or no prior anchor at all). Zeroing a fake segment can expose a new break further back, so this iterates to a fixed point.

Filter 3 — smash direction (`filter_smash_direction`)

After a detection in the smash zone (top 20% of frame) immediately followed by the shuttle going invisible (it exited the top of frame), any subsequent visible detection that lands *below* the top-20% re-entry zone before a genuine top re-entry is fake — almost always the hitter's racket head or body, picked up while the real shuttle is still above the camera.

Two guards prevent this from destroying real trajectory:

- `MIN_SMASH_GAP=4` — only fires if the shuttle was invisible for at least 4 frames first. A 1-3 frame gap means it barely left the frame and came straight back — real, not a fake mid-frame blob.
- **Serve-toss guard** (`SERVE_TOSS_SPEED_TH=5.0`) — if the cluster just before the smash-zone detection was nearly stationary (shuttle held for a serve), this isn't a smash exit at all; skip the filter entirely so the real serve trajectory that follows isn't zeroed.

Stage 4 — `TrackNetV3/fill_gaps.py`: filling real occlusion gaps

Input: `_ball_clean.csv` (+ optional `player_detections.csv`). **Output:** `_ball_filled.csv`, gap frames now interpolated (cubic spline, linear fallback), anchor frames never modified.

Why not just use InpaintNet's own gap-filling

InpaintNet's own mask generator only checks `Y[before] > threshold AND Y[after] > threshold` — "shuttle in-court." But "shuttle in-court" is not the same as "a rally is happening": right after a rally ends, the shuttle is still physically in-court while being picked up or held for the next serve — both anchors pass that check, so InpaintNet incorrectly bridges the *between-rally* gap as if it were a mid-rally occlusion. This file adds two gates InpaintNet lacks, plus several more found necessary in practice, before filling anything.

The gates, in order

A gap is filled only if **all** of these hold:

1. **Both anchors visible and in-court** (`Y > y_threshold`; the threshold is `0.0`, i.e. disabled, by default — the other gates already prevent between-rally fills, and a non-zero Y threshold incorrectly rejects legitimate anchors during high smashes/clears where Y genuinely approaches 0).
2. **Scene mask** — both players detected (YOLO confidence `> PLAYER_CONF_TH=0.5`) at both anchor frames. During pickup/service prep, typically one player is walking or crouching → `mask=0` → not filled. Dilated by `FILL_SCENE_DILATION=10` frames first, since a fast smash can make YOLO drop a player for a few frames even mid-rally. Exception: the smash *before-anchor* check is skipped, since the hitter is mid-jump and often missed by YOLO regardless.
3. **Gap length within an adaptive limit** — `MAX_GAP_SMASH=90` frames (3s @ 30fps) if the anchor is in a high smash (top 20% of frame, or moving upward fast: `dy < -8px` and speed `> 8px/frame` in the frame just before), else `MAX_GAP_OCCLUSION=25` frames (0.83s) for a regular occlusion.
4. **Implied speed sane** — straight-line distance between anchors divided by frame gap must be `<= MAX_FILL_SPEED=120 px/frame`. A real shuttle at ~300km/h in a 1920px-wide frame at 30fps is ~130px/frame, so anything implying more than that is two anchors from *different* rallies, not one continuous flight.
5. **Anchor-before trajectory quality** — the frames just before the gap must show real motion (`TRAJ_WINDOW=5` frames, need `>= 3` visible with mean speed `> MIN_ANCHOR_SPEED=2.0 px/frame`), not an isolated single-frame detection or a stationary shuttle. Relaxed to 1 visible frame for a smash-start anchor — a genuine high smash can leave only one frame of trajectory before the shuttle exits above the camera, with no earlier history to measure a speed from.
6. **Anchor-after resume speed** (non-smash gaps only) — the first visible frame *after* the gap must itself be moving at `>= MIN_RESUME_SPEED=5.0 px/frame`. Blocks fills into a slow/stationary between-rally cluster even when everything else looks plausible.

The smash-reentry pre-check

Separate from Filter 3 above (same idea, applied here because gap-filling sees pre-zeroed data Filter 3 already touched). If the anchor before a gap is in the smash zone and the gap is long enough (or the implied speed to `anchor_after` is impossibly high), scan forward and zero any detection that doesn't land back in the top-20% re-entry zone, extending the gap's real end to the genuine re-entry point before interpolating.

Interpolation

Cubic spline (`scipy.interpolate.CubicSpline`) using a small context window around the gap, if `gap_len >= 3` and at least 4 known points are available — falls back to pure linear otherwise. If a cubic fit overshoots frame bounds (a wild anchor context from a fast smash can force the spline outside `[0, width/height)`), the gap is retried with linear interpolation; if that *also* overshoots, the gap is left unfilled rather than writing an impossible position.

Parameters reference

filter_trajectory.py

Parameter	Default	Meaning
SPEED_BOTH_TH	130 px/frame	Bilateral spike threshold (Filter 1)
SPEED_HARD_TH	300 px/frame	One-directional impossible-teleport threshold
SPEED_INTERNAL_TH	300 px/frame	Impossible internal cluster jump (Filter 2)
ENTRY_TH	100 px/frame	Entry speed confirming a fake segment
SMASH_Y_FRACTION	0.20	Top fraction of frame = smash zone
SMASH_REENTRY_FRAC	0.20	Re-entry must land in top fraction
MIN_SMASH_GAP	4 frames	Min invisible frames before Filter 3 fires
SERVE_TOSS_SPEED_TH	5.0 px/frame	Stationary-cluster threshold → skip Filter 3

fill_gaps.py

Parameter	Default	Meaning
MAX_GAP_SMASH	90 frames (3s)	Max fillable gap after a high smash
MAX_GAP_OCCLUSION	25 frames (0.83s)	Max fillable gap for regular occlusion
MAX_FILL_SPEED	120 px/frame	Max implied speed between anchors
PLAYER_CONF_TH	0.5	YOLO confidence for the scene mask
FILL_SCENE_DILATION	10 frames	Scene-mask gap bridging
TRAJ_WINDOW / MIN_TRAJ_VISIBLE	5 / 3	Anchor-before quality check window/threshold
MIN_ANCHOR_SPEED	2.0 px/frame	Anchor-before must be genuinely moving
MIN_RESUME_SPEED	5.0 px/frame	Anchor-after must resume at real speed
SMASH_REENTRY_MIN_GAP	15 frames	Min gap before the reentry pre-check fires

Stage 3 — Player Detection

File: `analysis/player_detection.py`.

Input: the video + (optionally) `court.json`. **Output:** `player_detections.csv` — `frame_no`, `player_1_x`, `player_1_y`, `player_1_conf`, `player_2_x`, `player_2_y`, `player_2_conf` — plus a matching `.json` with full bounding-box detail, and (unless `--no_video`) an annotated `detection_output.mp4`.

This is the schema every later stage expects (`win_predictor/predict.py`, `rule_engine.py`, `TrackNetV3/fill_gaps.py`'s scene mask, `detect_rallies.py`'s scene mask) — any replacement detector has to produce exactly these column names to slot in.

How it works

Detection (`PersonDetector`): YOLO11 (`ultralytics.YOLO`, default weights `yolo11x.pt` — the largest/most accurate variant; smaller `yolo11n/s/m/l.pt` trade accuracy for speed), restricted to the `person` class (`classes=[0]`), confidence threshold `0.4`.

Court filtering (`DetectionFilter`): three checks per detection — confidence `>= min_conf` (0.4), bounding-box height `>= min_height` (60px — filters out small/distant false positives like a linesperson or ball boy in the background), and the box's **bottom-center point** (not its centroid) falls inside the court polygon. Bottom-center is deliberate — it approximates where a standing person's feet touch the ground, which is the meaningful "is this player on the court" test; a bounding box's geometric center is higher up (torso height) and would misjudge players near the court edge.

Court source: `--court_file` (the same `court.json` every other stage uses, rescaled to the video's actual resolution via `court_utils`), or `--no_court_filter` (keep every YOLO person detection unconditionally — needed for non-interactive/scripted runs where no court file exists yet), or (neither given) an interactive OpenCV window opens for manual 4-point click-annotation on the first frame — a fallback path, not the one `run_full_pipeline.py` actually uses (it always has a `court.json` by this point).

Per-frame output: whichever ≤ 2 filtered detections exist in a frame become `player_1` / `player_2`, in whatever order YOLO returned them. **This assignment is not stable across frames or semantically meaningful** — it's just "detection index 0" and "detection index 1" for that frame. Figuring out which slot consistently means "the far player" vs "the near player" happens **later**, in `win_predictor/rule_engine.assign_player_sides()` (see `06_win_prediction.md`) — by averaging each slot's Y position across the whole match and comparing to the near baseline. This two-step design (detect first, assign sides later from aggregate statistics) is simpler and more robust than trying to track player identity frame-to-frame during detection itself, which would need its own tracker and failure modes.

Parameters reference

Parameter	Default	Meaning
<code>--model</code>	<code>yolo11x.pt</code>	YOLO11 weights; smaller = faster, less accurate
<code>conf_thresh</code> (detection)	0.4	YOLO confidence to keep a raw detection
<code>min_conf</code> (filter)	0.4	Confidence gate in <code>DetectionFilter</code>
<code>min_height</code>	60px	Minimum bounding-box height
<code>--no_video</code>	off	Skip writing the annotated detection video (faster)

Why this file specifically (design note)

The whole file is deliberately simple relative to a "real" multi-object tracker: no identity tracking, no re-identification model, no motion model. Given the downstream need is just "two (x, y) points per frame, later sorted into far/near by aggregate position," a per-frame detector plus a separate side-assignment pass is enough, and avoids taking on a tracker's own failure modes (ID switches, track loss on occlusion) for information the pipeline doesn't actually need frame-to-frame.

Stage 5 — Rally Detection

File: `analysis/detect_rallies.py`.

Input: the gap-filled shuttle CSV + `player_detections.csv` (+ optionally the court-presence CSV from broadcast mode). **Output:** `<name>_rally.csv` —

`Start_Frame, End_Frame, Start_Time_sec, End_Time_sec, Duration_sec, Start_Time, End_Time`, one row per detected rally.

The core idea

A rally is "the shuttle moving with real speed, for a sustained stretch, while both players are actually on screen." The state machine (`segment_rallies`) tracks two states — not-in-a-rally, in-a-rally — and moves between them based on shuttle speed and visibility, with the scene mask pre-applied so frames where the camera isn't even on the match don't count at all.

Pipeline, in order

1. **Scene mask** (`generate_scene_mask` + `_dilate_mask`) — both players detected with confidence `> 0.5`, dilated by `SCENE_MASK_DILATION=100` frames. That's a large dilation, deliberately: a high smash can make YOLO miss the jumping hitter for 50-90 frames, and with a smaller dilation (45 was tried and found insufficient) those gaps weren't bridged — `apply_scene_mask` zeroed the pre-smash shuttle data, making the rally appear to start 2-3 seconds late. Between-rally periods have both players standing and detected throughout, so this large dilation doesn't falsely bridge across a real gap.
2. **Court mask** (broadcast mode only, `--court_mask_csv`) — ANDed with the scene mask, so a frame only counts if both the players *and* the main broadcast camera angle are present (see [02_court_presence_and_segmentation.md](#)).
3. **Apply the combined mask** — any frame that fails it gets its shuttle data zeroed (`X=0, Y=0, Visibility=0`), same as an undetected frame.
4. `segment_rallies()` — the state machine (below).
5. `filter_pickup_rallies()` — drop false positives.
6. **Duration filter** — drop anything `< MIN_RALLY_DURATION_SEC=2.0` (shortest real ground-truth rally observed was 4s; 2.0s comfortably removes 1-2s noise blips without risking a real rally).
7. `merge_close_rallies()` — residual safety net for splits `fill_gaps` didn't catch.
8. `apply_lookback()` — recover the true rally start, which the state machine's `MIN_START_VISIBLE` requirement systematically delays.

The state machine

Starting a rally: need `MIN_START_VISIBLE=3` consecutive frames with `Visibility==1` and `speed > START_SPEED=3.0 px/frame`. Speed is explicitly zeroed at any invisible → visible transition — a `diff()` from `(0,0)` to a real position produces a huge fake speed that must not count as "the shuttle moving." A duplicate-position guard also carries the last real speed forward across frames where X/Y repeat exactly (a tracking artifact that would otherwise register as `speed=0` mid-rally).

Ending a rally, two independent conditions:

- **Gap-based:** `END_GAP_FRAMES=10` consecutive invisible frames.
- **Stationary-based:** an N-of-M window (`STATIONARY_WINDOW=10`, `STATIONARY_MIN_SLOW=7`) where at least 7 of the last 10 *visible* frames are slow (`speed < STATIONARY_SPEED_TH=3.0`) and below the smash apex (`Y > STATIONARY_MIN_Y=50`) — filters out a slow-motion interpolation artifact near the top of frame from being mistaken for a genuine ground landing). N-of-M instead of a simple consecutive counter because real "stopped" speed jitters around the 3.0 threshold — a strict consecutive-frame counter would keep resetting on noise.

Lookahead before confirming an end (`END_LOOKAHEAD_FRAMES=15`): peek 15 frames past a candidate end condition — if fast movement resumes within that window, the rally hasn't actually ended (a brief tracking dropout, not a real end); clear the end condition and keep going instead of splitting one rally into two.

`filter_pickup_rallies()` — floor-pickup and static-noise rejection

Two independent checks on a candidate rally's first second:

- **Pure noise:** if total 2D travel in that window is `< MIN_NOISE_TRAVEL_PX=15` and duration `< 4.0s`, it's a static blob, not a rally — drop it.
- **True pickup:** if the shuttle starts within `MAX_FEET_DIST_PX=40` of whichever player is nearer (vertically, `-50 < Δy < 40`, and horizontally, `Δx < 150`), and horizontal travel stays under `MAX_PICKUP_X_TRAVEL=80` — a floor pickup lifts the shuttle mostly vertically at the feet, unlike a real serve or rally shot — drop it. Only applied to candidates under 4s (a real pickup never lasts that long).

`apply_lookback()` — recovering the true start

`MIN_START_VISIBLE=3` means the state machine's detected start is always *at least* 3 frames later than the shuttle actually started moving — worse if the first few frames of real motion happened to have gaps. This walks backward from the confirmed start (up to `LOOKBACK_MAX_FRAMES=150`, capped at the previous rally's end so lookback can never steal frames from an adjacent rally), looking for a "rest period" — `LOOKBACK_REST_MIN_LEN=5` consecutive frames that are either invisible or low-confidence (`< LOOKBACK_REST_CONF_TH=0.40`) — and moves the rally's recorded start to just after that rest period, i.e. back to where the shuttle first stirred out of its pre-serve stillness.

`merge_close_rallies()` — residual split repair

Most rally-splitting occlusion gaps are already fixed at the `fill_gaps.py` level (see [03_shuttle_tracking.md](#)); this is a lightweight safety net for whatever slips through — very brief net-exchange dropouts under a second. Two consecutive detected rallies merge if the gap between them is `<= MERGE_GAP_SEC=0.75s`, with a wider allowance (`<= 4.0s`) specifically for a smash-exit case: the previous rally was short (`<= 2.0s`) and ended with the shuttle high in frame (`Y < 250`) — i.e. it plausibly exited camera view mid-smash rather than genuinely ending.

Parameters reference

Parameter	Default	Meaning
<code>START_SPEED</code>	3.0 px/frame	Min speed to begin a rally
<code>MIN_START_VISIBLE</code>	3 frames	Consecutive fast frames to confirm a start
<code>END_GAP_FRAMES</code>	10 frames	Invisible frames before ending (gap-based)
<code>END_LOOKAHEAD_FRAMES</code>	15 frames	Peek window before confirming an end
<code>SCENE_MASK_DILATION</code>	100 frames	Bridges brief YOLO misses during smashes
<code>MIN_RALLY_DURATION_SEC</code>	2.0s	Drop anything shorter after detection
<code>STATIONARY_SPEED_TH / _WINDOW / _MIN_SLOW</code>	3.0 / 10 / 7	N-of-M stationary-shuttle end condition
<code>STATIONARY_MIN_Y</code>	50px	Excludes smash-apex false stationary reads
<code>MIN_NOISE_TRAVEL_PX</code>	15px	Static-noise rejection threshold
<code>MAX_FEET_DIST_PX / MAX_PICKUP_X_TRAVEL</code>	40 / 80px	Floor-pickup rejection thresholds
<code>LOOKBACK_MAX_FRAMES</code>	150 (~5s)	Max backward search for true rally start
<code>LOOKBACK_REST_CONF_TH / _MIN_LEN</code>	0.40 / 5	Rest-period detection for lookback
<code>MERGE_GAP_SEC</code>	0.75s (4.0s for smash exits)	Residual split-merge threshold

Stage 6 (part 1) — Win Prediction

Files: `win_predictor/predict.py`, `win_predictor/rule_engine.py`,
`win_predictor/traj_rf.py`, `win_predictor/config.py`.

This is the most complex stage in the pipeline — three separate prediction mechanisms (a hand-written rule engine, a CNN video classifier, a trajectory Random Forest) whose outputs get fused per-rally into a single call. It's worth understanding *why* three mechanisms exist rather than just the most accurate one: they were built in that order, over time, each added because the previous one(s) had a specific, measured blind spot, and the fusion logic is largely "trust whichever one is actually strong for this kind of rally."

Input per match: `<name>_ball_filled.csv`, `<name>_rally.csv`, `<name>_court.json`,
`player_detections.csv`, optionally the video itself (needed for CNN clips) and
`<name>_shuttle_size.csv` (needed for the RF's size features).

Output: `win_predictions.csv` — one row per rally: `rally_idx`, `start_frame`, `end_frame`,
`start_time`, `end_time`, `winner`, `loser`, `win_reason`, `reason_src`, `fault_side`, `fault_src`,
`cnn_class`, `cnn_conf`, `cnn_oob`, `cnn_hn`, `cnn_wbl`, `score_<player_a>`, `score_<player_b>`.

`win_reason` is always one of exactly three classes (`config.CLASSES = ['out_of_bounds', 'hits_net', 'wins_by_landing']`) — this fixed 3-way schema is what the whole downstream fusion/report/webapp layer is built around.

Court geometry setup (`rule_engine.load_court_from_json`)

Reads `court.json`, derives everything the rest of the file needs:

- `court_poly_cv` — the 4 corners as an OpenCV-ready polygon, for `cv2.pointPolygonTest` in/out checks.
- `net_top_Y` (cable) and `net_ground_Y` (floor reference) kept **separate** — `net_top_Y` collapses to nearly the far baseline's pixel position in this camera's perspective, so depth alone can't reliably distinguish "near the net" from "near the far baseline"; `net_ground_Y` plus the net's actual X span (narrower than full court width — the net posts sit inside the doubles sidelines) is the reliable "near the net" test.
- `net_tol_px = 720 * 0.02 ≈ 18px` — a shared tolerance used throughout: a line belongs to the court/net it bounds (the actual badminton rule), so a point within `net_tol_px` of a boundary still counts as in/at it, not past it.
- `H_metric` — homography from image pixels to a top-down metric rectangle (singles: 5.18m × 13.4m), used everywhere a "how far outside the court, in real distance" answer is needed instead of a pixel-space guess.

Player side assignment (`assign_player_sides`)

Solves the problem left open by player detection (see [04_player_detection.md](#)): `player_1` / `player_2` are arbitrary per-frame detection slots, not semantically "far" or "near." Averages each slot's Y position across the whole match, compares distance to the near baseline (`court_bottom_y`) — whichever slot is *closer* on average is NEAR (Player B); the other is FAR (Player A). One assignment per match, computed once, not per-frame — a player doesn't switch court ends mid-match.

The rule engine (`analyze_rally_end`)

For each rally's `end_frame`, this looks at the shuttle's last ~3 seconds (`ANALYSIS_SEC=3.0`) of trajectory and works through checks in **priority order**, returning as soon as one fires:

```
CHECK 0 Shuttle stationary      (most reliable — the shuttle visibly stopped)
CHECK 1 Cable crossing          → wins_by_landing
CHECK 2 At cable + stopping     → hits_net
CHECK 2b Clear positional baseline OOB
CHECK 3 OOB via extrapolation
CHECK 4 Baseline proximity
CHECK 5 Above cable, in court   → wins_by_landing
CHECK 6 OOB fallback
CHECK 7 Final fallback         → wins_by_landing
```

Why this order: each check is progressively less direct evidence. "The shuttle visibly stopped and its position tells you everything" (CHECK 0) is unambiguous when it's available (real trajectory data very often loses the shuttle right around actual ground contact — see the discussion in this doc's "known limitation" section below — so CHECK 0 fires less often in practice than its priority ranking suggests). Further down, checks rely on increasingly indirect signals: extrapolating a still-moving trajectory forward to guess where it would have gone (CHECK 3), or falling back to whichever side of the net the last few frames happened to be on (CHECK 7) when nothing more specific matched.

CHECK 0 — stationary (`static = vels[-6:]` where `speed < 15px/frame`, need `>= 3` such samples): average their position, test against the court polygon (with `net_tol_px` tolerance) and net proximity. This is the highest-confidence path when the shuttle is genuinely at rest.

CHECK 1/2 — cable crossing, via the *final segment only*. `_last_hit_index()` finds the most recent sharp direction change in *smoothed* velocity (3-frame rolling average, so a slow net tap's direction change isn't lost to pixel jitter, `angle_thresh_deg=60`) — this isolates the shuttle's motion since the last racket contact, so an earlier net exchange within the 3-second window can't be mistaken for the final one. If the final segment crosses the cable height: distinguish a clean winner that briefly touches cable-height in passing from a genuine net fault by checking whether the trajectory *lingered* near cable height (`frames_near_cable >= max(8, 40% of segment)`) rather than passing through briefly — a real net fault drags along the tape, a clean net shot doesn't.

CHECK 3 — extrapolation (`_extrapolate_exit`): if the trajectory hasn't decelerated much (`decel_rate < 0.35`), project it forward at constant velocity, frame by frame, up to 45 frames, until it exits the court polygon. Guarded against a specific false-positive: if velocity points backward through frames that were actually a *different*, earlier shot within the analysis window, this would extrapolate through the net in the wrong direction — detected and skipped (`_backward_extrap`).

CHECK 7 — final fallback, when nothing else matched. This is most often a soft, dying net shot that never registered enough speed or deceleration to trip the explicit net checks. Only called a clean winner if the shuttle actually ended up away from the net's floor footprint *and* its trajectory reached cable height at some point — if it landed far from the net or never approached the cable at all, it's scored as a winner by default rather than a fault.

Fault-side determination

Separate from *which event* happened is *whose fault* it was — which player hit the shuttle last.

`_oob_fault_side()` uses which of the 4 boundary lines was crossed (past the near baseline → far player's fault; past the far baseline → near player's fault; a sideline exit is split by net-depth).

`hits_net` fault side comes from **approach velocity** just before the tape contact (which direction was it moving when it hit the net), not landing position — because a net fault can bounce back onto either side, making landing position an unreliable signal for who caused it.

Post-processing overrides (`apply_overrides`)

A cascade of narrower corrections applied *after* the main check, each addressing one specific measured failure mode found in evaluation — named so they're traceable in `reason_src` /debug output: `clip` (vertical net-clip shape check — mostly-vertical fall + deceleration + resting near the net, even if the main checks missed it), `prox` (landing lands inside the net's height band), `oob-fault` (recompute fault side from final position once event is confirmed OOB), `stable` / `stable2` (shuttle demonstrably stopped well inside the court), `arc` (steep downward arc mid-flight, still inside), `netfreeze` (shuttle froze at the net base for many frames — different signal than a single stationary check), `plhs` (player-proximity correction — which player was actually moving toward the shuttle, as a fault-side tiebreaker), `cpf` (court-partition fault using the net's floor line as a depth divider).

A few explicitly **rejected** alternative designs are left as comments in the code, with their A/B test results, specifically to stop them being re-tried: extending the analysis window forward past `end_frame` (tested at 1.0s and 1.5s, both regressed win-reason accuracy 64.9% → 53-55%, because a shuttle that's genuinely gone out keeps rolling further away — "looking further forward" gives a *worse* read on where it landed, not better); using last-hit-segment velocity direction instead of `_fault_side()` for OOB fault (regressed 58.0% → 53.4%).

The CNN ensemble (`predict.py`)

Architecture: `r2plus1d_18` (torchvision video backbone), with the final layer replaced by a small classifier head (`BatchNorm1d` → `Linear(256)` → `ReLU` → `Dropout(0.3)` → `Linear(3)`), predicting the 3 `CLASSES` directly from video.

Two models, two temporal windows, averaged: a 20-frame clip sampled from a 1.5s window (`SEQ_LEN_20F=45` frames back) and a 16-frame clip from a 3.0s window (`SEQ_LEN_16F=90` frames back) — both centered on the shuttle's last known position (extrapolated forward along its recent velocity if tracking was lost a few frames early, up to 15 frames of extrapolation), cropped to `CROP_FRAC=44%` of frame height around that point, resized to 112×112. Two temporal scales because a genuine landing/fault event has both fast local dynamics (contact) and slower context (the approach) — the ensemble averages both models' softmax outputs.

Confidence: `cnn_conf = max(softmax)`, `cnn_class = argmax`.

The trajectory Random Forest (`traj_rf.py`)

A meta-learner: **not** trained on raw video, but on 24 hand-engineered features from the shuttle trajectory + the CNN's own probabilities. Binary classifier — OOB vs WBL only (`hits_net` is handled entirely by the CNN threshold gate below, never by the RF).

Feature groups:

- **Trajectory (11):** `margin_cm` (real-world distance from the nearest court boundary, via the same metric homography), average/absolute velocity, a stationary-pattern flag (`is_post`), point density near the end (`n_tight`), a fitted parabola's curvature and vertex-extrapolation distance (`para_a`, `para_extrap` — from `np.polyfit` on the last 25 frames), distance from the far baseline, positional variance (`std_y`).
- **Shuttle apparent size (6):** mean/trend/ratio of the shuttle's detected pixel size near the end, from a separate `_shuttle_size.csv` — a shuttle growing larger as it approaches the camera/ground is a physical landing signal orthogonal to position.
- **CNN probabilities (3):** `prob_oob`, `prob_hn`, `prob_wbl` fed in directly — the RF can learn to trust or distrust the CNN differently depending on the trajectory context.
- **Net geometry (4):** normalized last position, distance to the net's floor line, whether the last X position is within the net's horizontal span.

Fusion strategy — which prediction wins

Per rally, in order:

1. **HN gate first.** If `cnn_hn >= HN_THR (0.60)` — or the lower `NET_ZONE_HN_THR (0.45)` when the shuttle's last position is both within the net's X span and within `NET_ZONE_DIST=40px` of its floor line — the CNN's "hits net" call is trusted, **unless** the RF disagrees with high conviction: **Strategy E** — if the RF says WBL and the shuttle's `margin_cm` is positive (genuinely inside the court), the HN call is overridden to WBL (`reason_src = ML_STRAT_E`). Otherwise, `hits_net` stands (`ML_HN`, or `ML_HN_I` when the lowered net-zone threshold was what triggered it).
2. **Otherwise, if the RF has a prediction** (OOB or WBL): normally trusted directly (`ML_RF`) — **unless Strategy F** fires: RF says WBL, but the *rule engine* independently says OOB, **and** the trajectory shows fast lateral motion (`abs_vx > 2.0` and `speed > 4.0`) — a fast sideways shot is exactly the case where OOB is more likely correct than a size/position-based RF read, so the rule engine's OOB call wins here (`ML_STRAT_F`).

3. **No RF prediction available** (model not loaded, or not enough trajectory to build features): fall back to the rule engine's own `event` directly (`reason_src = RULE`).

Fault side, computed *separately* from `win_reason`: for `wins_by_landing`, directly from the median shuttle Y in the last 10 visible frames vs. `net_ground_Y` (`WBL_Y` — position at landing is the most direct evidence for who's side it landed on); for other reasons, the rule engine's own fault computation stands.

`CNN_FIRST_THR = 0.55` exists in config but the actual fusion logic above is the HN-gate/RF-first design (Strategies E/F/I/J) that superseded a simpler "trust CNN if confident enough, else rule engine" scheme — kept for backward compatibility/reference, not the active decision path.

Known limitation — the stationary-shuttle detector often finds nothing

CHECK 0 (the rule engine's most reliable path) needs the shuttle to be *tracked* while at rest. In practice, TrackNetV3 frequently loses the shuttle entirely right around actual ground contact — it was trained to spot a small fast-moving object against contrast, and a stationary shuttle sitting on a green court blends into the background. Verified on real match data: a rally's shuttle trajectory can be falling smoothly and steadily right up to the very last tracked frame (no visible slowdown at all), then tracking drops to `Visibility=0` and never resumes for the rest of the rally. When this happens, the `vels[-6:]` window CHECK 0 examines is really "the tail of a still-falling arc," not a genuine at-rest position — the resulting average lands a few frames *before* the true landing spot.

This is a structural limitation of the detector, not a tuning gap — you can't find a stationary shuttle in frames where it was never tracked. The improvement worth building (not yet implemented): extrapolate the last several *valid* tracked points' falling trend forward through the same per-rally homography until it crosses the ground plane, and use that projected crossing as the landing frame/position instead of whatever the tracker happened to see last. See the win-predictor Random Forest's `margin_cm`/net-geometry features for the existing homography-based real-world distance mechanism this would build on.

Parameters reference

Parameter	Default	File	Meaning
ANALYSIS_SEC	3.0s	rule_engine	Trajectory window examined before end_frame
net_tol_px	~18px (720×0.02)	rule_engine	Shared boundary tolerance
HIGH_SPEED / MED_SPEED	350 / 150 px/ frame	rule_engine	Speed classification thresholds
RELIABLE_SPD_MIN	40 px/frame	rule_engine	Min speed counted as a "reliable" velocity sample
VERTICAL_RATIO	1.5	rule_engine	_vertical_net_clip_override vy/vx ratio
VERTICAL_DECEL	0.25	rule_engine	Deceleration fraction for the net-clip override
COURT_W_M / COURT_H_M	5.18 / 13.4m	rule_engine	Singles court metric dimensions
CLIP_FRAMES	20	config	Frames sampled per CNN clip
SEQ_LEN_20F / SEQ_LEN_16F	45 / 90 frames	config	CNN look-back windows (1.5s / 3.0s)
CROP_FRAC	44% of height	config	CNN clip crop size
OUT_SIZE	112px	config	CNN input resolution
HN_THR / NET_ZONE_HN_THR	0.60 / 0.45	config	CNN hits-net probability gate (Strategy I)
NET_ZONE_DIST	40px	config	Net-zone proximity for the lowered HN gate
CNN_FIRST_THR	0.55	config	Legacy threshold, superseded by HN-gate fusion

Stage 6 (part 2) — Fusion, Commentary, and PDF Report

Files: `fusion_layer.py`, `run_unified_pipeline.py`.

Runs immediately after win prediction (Stage 6, part 1) finishes writing `win_predictions.csv`. `run_unified_pipeline.py` is the actual entry point `run_full_pipeline.py` calls for this whole step; `fusion_layer.py` is one piece it uses internally.

`fusion_layer.py` — merging with the shot classifier

Input: `win_predictions.csv` (canonical winner/reason/score — Stage 7 per the code's own naming, "Stage 7 is THE winner stage" is an explicit design directive preserved in a comment) + optionally a separate `*_extended.csv` shot classifier output (`find_extended_csv` picks the newest one present by file mtime).

Schema normalization (`_normalize_schema`): `predict.py`'s CNN-era output uses different column names (`winner`, `win_reason` as the raw class string) than an older "stage7" convention (`win_point_player`, `win_reason` as a full sentence like "opponent goes out of bounds"). Since `win_point_player` not being present is exactly the signal for "this needs normalizing," the function checks for that column first and passes through unchanged if it's already in the older format — supporting both without needing to know in advance which one it's looking at. Also derives `detection_confidence` (`HIGH >= 0.8, MEDIUM >= 0.6, else LOW` from `cnn_conf`) and picks the two `score_*` columns positionally for `roundscore_A` / `roundscore_B`.

Merging: if no shot-classifier CSV exists, "degraded mode" — the merged view is just Stage 7's own data, `shot_source="stage7"`. If one does exist, its `ball_types` (a real shot-type classification — smash, net shot, clear, etc.) replaces Stage 7's own trajectory-based guess, and a `prediction_agrees` column records whether the two systems' independent winner calls match — useful as a cross-check, not something later code branches on.

Two output files: `merged_analysis.csv` (everything, plus the agreement columns) and `badminton_analysis_enhanced.csv` — an externally fixed 8-column schema (`start_time`, `end_time`, `win_point_player`, `win_reason`, `ball_types`, `lose_reason`, `roundscore_A`, `roundscore_B`) that a separate downstream viewer (the "BadmintonAnalysis" Streamlit app, referenced but outside this pipeline's scope) consumes directly.

`run_unified_pipeline.py` — the actual orchestration

Four phases, in order:

[1/4] Win Predictor. `ensure_win_predictions()` — runs Stage 6 part 1 if `win_predictions.csv` doesn't already exist (same skip-if-done pattern as everything else).

[2/4] Data fusion. Calls `fusion_layer.DataFusionLayer` as above, saves both output CSVs.

[3/4] AI match commentary. Two-layer design: a **deterministic** factual summary computed entirely in code (`stats_summary`), which an LLM is then asked to *restyle* — never to invent or recompute any fact.

`stats_summary(df, player_a, player_b)` computes, purely from the merged data: total rallies, per-player win counts and each player's single most common win reason (with count), the single longest rally (via `_rally_seconds` on each row, `idxmax`), that rally's own winner/reason/ final shot type, the single most common rally-ending shot type overall, and the final recorded score. All of it assembled into one paragraph — this exact text is also the **fallback** output if the LLM step fails or is unavailable.

`llm_commentary(df, player_a, player_b)` sends that factual paragraph, plus optional player background text (`load_player_context` — matches player names against `.txt` bio files by token overlap, truncated to a sentence boundary) to a local LM Studio instance (Phi-3, via a plain HTTP request — `import requests` is local to this function since it's an optional dependency), with an explicit instruction set: keep every number, name, rally number, and score *exactly* as written, rewrite style only. Falls back to the raw `stats_summary` text (`source='STATS'`) if the LLM call fails for any reason — the report is never blocked on an external service being reachable.

Zero-rally guard. `stats_summary` / `build_match_semantics` / `llm_commentary` all assume at least one rally exists — `idxmax()` on an empty Series, `.iloc[-1]` on an empty frame, `value_counts().idxmax()` on an empty result all raise. A genuinely rally-free clip (wrong video, a slice that's all pre-match footage, the camera never actually on the court) is a real possibility with user-uploaded input, not just a theoretical edge case — found and fixed via the web app's live testing (see [09_web_review_app.md](#)). `run_unified_pipeline` checks `len(merged) == 0` before calling into the summary functions at all and substitutes a plain "No rallies were detected..." message (`source='STATS'`) instead of letting the pipeline crash on the last step.

[4/4] PDF report. `pick_highlights(df, n=5)` — top-5 rallies by duration (parsed from `start_time` / `end_time` strings), sorted back into chronological order for the summary. `generate_pdf()` hands off to an external `BadmintonAnalysis` project's `pdf_generator.generate_pdf_report` (outside this pipeline's own code, loaded via a `sys.path` insert) with the player names, the 5 highlights, and the final commentary text.

Written outputs (`<match>/unified/`): `merged_analysis.csv`, `badminton_analysis_enhanced.csv`, `analysis_report.txt` (player names + source + generation timestamp header, then the commentary text), `<players>_badminton_highlights_<timestamp>.pdf`.

Parameters / config reference

Item	Value	Meaning
Highlight count	5	Top-N rallies by duration selected for the PDF
LLM rewrite rule	facts frozen	Every number/name/score must appear unchanged in the LLM's output
Commentary fallback	<code>stats_summary</code> <code>text</code>	Used whenever the LLM call fails or is unreachable
<code>source</code> field	'LLM' or 'STATS'	Recorded in the report so it's clear which path produced the text

Stage 7 — Verification Video

File: `analysis/annotate_video.py`.

Input: the video, shuttle CSV, player CSV, court JSON, optionally the court-presence CSV and the rally CSV. **Output:** `<name>_annotated.mp4` — one combined visual sanity check: court boundary + net line + both players + shuttle trail, all drawn on the actual footage, real H.264 so it plays in a browser.

Why it exists

Every other stage's output is a CSV — useful for the algorithms downstream, useless for a human trying to spot-check whether the pipeline actually got something right. This renders everything at once onto one video so a human reviewer can watch it and immediately see, e.g., the court boundary drawn in the wrong place, a player marker drifting off a person, or a rally that visibly happened with no detection.

Two purpose-built modes

`--rallies_only` — trimmed and joined: skip straight to the frame windows from `<name>_rally.csv` (padded `±rally_pad_sec`, default 1.0s), concatenated into one short output. Fast, good for checking each *detected* rally really is one continuous rally. Structural blind spot: a rally the model missed entirely is just silently absent — there's nothing to see.

`--mark_rallies` — full-length, untrimmed: every frame of the original video is kept, with a "RALLY DETECTED" marker (thick green border + label) drawn only during frames inside a detected rally window. Scrubbing the whole thing and finding a real rally playing out *without* the marker on screen is a missed detection — structurally impossible to see with `--rallies_only`. This was an explicit choice: the model isn't 100% accurate, and a trimmed video makes a missed rally invisible by construction rather than something a reviewer can notice. `run_full_pipeline.py` uses this mode by default.

Mutually exclusive — both modes need `<name>_rally.csv` to exist first.

Overlay gating — court/net only where the court actually is

If a court-presence CSV exists (broadcast mode), the court/net overlay is only drawn on frames where `court_present == 1` — otherwise, since `--mark_rallies` renders the *whole* video including intro/crowd/replay cutaways, the court outline would be superimposed over footage where the real court genuinely isn't there, which is actively misleading rather than just imprecise. If no presence CSV exists (a pre-trimmed clip, not `--broadcast`), the overlay draws on every frame — the correct assumption there, since the whole clip *is* court footage. Verified directly: a synthetic test with a known presence mask showed the overlay's corner markers present exactly on the marked-present frames and absent elsewhere, byte-for-byte.

Rendering

For each frame in the active window(s): court polygon (`cv2.polylines` + corner dots) and net line, both players (colored dots — blue = the "P1" detection slot, orange = "P2", **not** semantically far/near, same caveat as [04_player_detection.md](#) — this is a raw visualization of the detector's slots, not the rule engine's far/near assignment), a shrinking-radius shuttle trail (last `--trail` frames, default 10, skipping invisible ones), the rally marker if applicable, and a frame counter label.

Output codec — the one non-obvious part

`cv2.VideoWriter` only reliably writes `mp4v` (MPEG-4 Part 2) in this environment. That plays fine in VLC/most desktop players and in OpenCV itself, but **no modern browser supports it in a `<video>` tag** — only H.264/VP9/AV1. A file written directly with `mp4v` would just show as a black box with no error in the web review app.

Fixed by writing to a temporary file with `cv2.VideoWriter` as before, then transcoding to the real output path via `ffmpeg -c:v libx264 -preset fast -crf 23 -pix_fmt yuv420p` (the same encoding convention `video_utils.py` uses elsewhere), and deleting the temp file. Verified directly: `ffprobe` on a regenerated file confirmed `codec_name=h264` where it previously reported `mpeg4`.

Parameters reference

Parameter	Default	Meaning
<code>--trail</code>	10 frames	Shuttle trail length
<code>--rally_pad_sec</code>	1.0s	Padding around each rally window (<code>--rallies_only</code> only)
Output codec	H.264, CRF 23	Browser-compatible, via a post-write ffmpeg transcode

Web Review App

Files: `webapp/jobs.py`, `webapp/server.py`, `webapp/static/index.html`, `webapp/static/app.js`.

A local Flask + vanilla-JS app: upload a video in the browser, watch the real pipeline (`run_full_pipeline.py`) process it with live progress, then review the actual detected rallies against the actual annotated video and correct anything wrong — without ever modifying the pipeline's own output files.

Run it:

```
cd pipeline_release
python3 webapp/server.py --port 8000
```

`jobs.py` — running the real pipeline as a background job

Does **not** reimplement any pipeline logic. It shells out to the exact same `run_full_pipeline.py` a terminal user would run, and gets live progress by parsing its existing stdout — no changes needed to the pipeline script itself.

One job at a time, by design: a single background worker thread pulls job IDs from a `queue.Queue`. The pipeline stages are GPU/CPU-heavy, and manual court annotation briefly needs an exclusive port (5050) for `annotate_court.py`'s own browser tool — concurrent jobs aren't something this local, single-user tool is built to support.

Progress parsing

`run_full_pipeline.py` already prints structured step headers for every stage: `"2b/7 Court-presence mask"`, etc. (see `00_overview.md`). `STEP_RE = re.compile(r'^\s*([0-9]+[a-z]?/7)\s+(.+?)\s*$')` matches these directly from the subprocess's live stdout, updating `job.current_step` and appending to `job.steps_seen` as they arrive. `SKIP_RE` and `DONE_RE` catch the `[skip]` lines and the final `DONE -- <path>` line the same way.

A real bug found via live testing: the child process's stdout is piped (not a TTY), so CPython defaults to fully block-buffered output — `run_full_pipeline.py`'s own `print()` calls, and everything it in turn shells out to (TrackNetV3, YOLO), sat in an internal buffer and never reached the parser until a buffer's worth accumulated or the process exited. Fixed by setting `PYTHONUNBUFFERED=1` in the **environment** passed to the top-level `subprocess.Popen` call — not just a `-u` flag on that one process, since the environment variable is what actually propagates down through every further `subprocess.run()` call `run_full_pipeline.py` makes internally (which inherit the parent's environment by default, but would *not* inherit a `-u` flag applied only to the immediate child's own invocation). Verified: before the fix, `steps_seen` stayed empty for the entire run despite the subprocess doing real, visible work; after, individual steps and their live sub-process output (`Using device: cuda`, frame counts, etc.) streamed through in real time.

The manual-annotation wait state

If `--court_mode manual` is used, the pipeline blocks on `annotate_court.py`'s own browser server until a human clicks 4 points — correctly so, but a naive progress parser would just show the job "stuck" with no explanation. Detected by step *label* (`MANUAL_ANNOTATION_HINT = "Court + net annotation (manual)"` — deliberately by label, not by step ID, since the ID `"2/7"` is shared with the automatic-mode label too), setting `job.status = "waiting_for_annotation"`. The frontend shows explicit instructions to open port 5050 rather than a spinner with no explanation.

Retry — reusing the pipeline's own resumability

`POST /api/jobs/<id>/retry` on a failed job creates a **new** job with the **same** `match_name` as the failed one (not a fresh sanitized name). This matters: `run_full_pipeline.py` already skips any stage whose output file exists, so retrying with the same match name picks up exactly where it failed instead of redoing completed work. This was added after live testing hit a real failure partway through a run — a naive re-upload through the normal upload form would have gotten a *new*, differently-named match folder (the upload endpoint's collision-avoidance logic exists for a different reason — see below — and would have actively defeated the pipeline's own resumability here).

`server.py` — the Flask API

Route	Method	Purpose
<code>/, /static/<file></code>	GET	Serve the frontend
<code>/api/jobs</code>	POST	Upload a video, create + enqueue a job
<code>/api/jobs/<id></code>	GET	Job status (polled by the frontend every 2s)
<code>/api/jobs/<id>/retry</code>	POST	Re-run a failed job, same match name
<code>/api/jobs/<id>/data</code>	GET	Parsed rally data for the review screen
<code>/api/jobs/<id>/video</code>	GET	The annotated video (range-request enabled)
<code>/api/jobs/<id>/corrections</code>	GET/POST	Load/save human review edits
<code>/api/jobs/<id>/report.pdf</code>	GET	The generated PDF report

Match-name collision avoidance (upload only, not retry)

`_sanitize_match_name()` strips to `[A-Za-z0-9_-]`, and if a folder with that name already exists, appends `_2`, `_3`, etc. until it finds a free one. This exists specifically for the **upload** path — reusing an existing match folder there would make `run_full_pipeline.py` treat old, unrelated data as if it were the fresh upload's own output (every stage would just skip itself, "completing instantly" with someone else's numbers). Deliberately **not** applied to `/retry`, which needs the opposite behavior (see above).

`/data` — converting `win_predictions.csv` into the review schema

Reads the real CSV schema directly (see [06_win_prediction.md](#) for the source columns): `winner` / `loser` (real player-name strings) mapped to `'A'` / `'B'` by comparing against the job's own `player_a` / `player_b`; `win_reason` mapped to short internal codes (`out_of_bounds→oob`, `hits_net→net`, `wins_by_landing→land`); frame numbers converted to seconds via the video's actual fps (read fresh with `cv2.VideoCapture`, not assumed); score columns looked up by name with the same `[:8]`-truncation `predict.py` uses when writing them (`f"score_{player_a[:8]}"`), since a long player name gets truncated in the CSV's own column header. A rally is flagged `low-confidence` if `cnn_conf < 0.6` — reusing a real model confidence signal already present in the data, not a separate heuristic.

Corrections are layered on top, never overwriting the source. If `corrections.json` exists in the match folder, `/data` returns *its* rally list (a full snapshot, not a diff) instead of re-deriving from `win_predictions.csv`. The CSV itself is never written to — corrections live in a completely separate file, so the model's original output and a human's corrections both survive independently.

`/video` — range-request serving

`send_file(path, conditional=True)` — Flask/Werkzeug's built-in support for HTTP `Range` requests, required for a browser `<video>` element to seek/scrub through a large file without downloading all of it first. Verified directly: a `Range: bytes=0-1023` request against a real 1.25GB annotated video correctly returned `206 Partial Content` with a matching `Content-Range` header.

`/report.pdf`

Globs `<match>/unified/*.pdf` rather than reconstructing the exact filename — the real filename includes a generation timestamp (`<players>_badminton_highlights_<timestamp>.pdf`, see [07_fusion_commentary_pdf.md](#)), which this endpoint doesn't need to know in advance. Picks the newest by mtime if more than one exists.

Frontend — `index.html` + `app.js`

Three screens, one page, `goStep(n)` toggles visibility:

1. Upload — file picker (drag-drop or click-to-browse), player names, match name (optional), court-mode radio (auto/manual), broadcast checkbox. Submits via `XMLHttpRequest` (not `fetch`) specifically to get upload-progress events for large video files.

2. Processing — polls `GET /api/jobs/<id>` every 2 seconds. Renders the step checklist **directly from the server's** `steps_seen` array — not a hardcoded local list — since the real step sequence differs between direct and `--broadcast` mode (see [00_overview.md](#)) and the server is the only thing that actually knows which one is running. Shows the manual-annotation instructions or a failure box (with a **Retry** button, see above) based on `job.status`.

3. Review — fetches `/data` once processing completes, then:

- A real `<video controls preload="auto">` element (**not** a custom player — an earlier version had neither `controls` nor real preload, which rendered as a black box with nothing playable; found via direct user testing, not caught by any automated check).
- A rally timeline: draggable-edge blocks per rally, click a >14-second gap to insert a new rally there (`addRally`), click a block to select + seek the video to it.
- A rally-by-rally table: every field (start, end, winner, reason) directly editable inline, a live-computed running score column (recomputed client-side on every edit, not read from the server's stale `score_after`), a delete button per row, an explicit "+ Add missed rally" button independent of the timeline's gap-click affordance.
- A "How points were won" bar chart, computed client-side from whatever the current (possibly-edited) rally reasons are.
- A Match Report card — the same `analysis_report.txt` commentary text the PDF contains, plus a direct PDF download link.
- Save Draft / Confirm & Finalize, both `POST /api/jobs/<id>/corrections` with the full current rally list + comments text; the only difference is the `confirmed` flag.

Winner is computed, not independently settable

An earlier version had a separate match-level winner override (A/B toggle) *in addition to* the per-rally table — which could disagree with the rally tally, producing a confusing "your override disagrees with the score" warning. Removed: winner is now purely `aCount vs bCount` from the rally table, with a small note pointing back to the table as the only place to change it. One source of truth, not two that can drift apart.

Bugs found via real end-to-end testing (not caught by unit checks)

Each was caught because the app was run against a **real** upload through the real pipeline, not mocked data — worth listing because they're the kind of bug that only shows up under real conditions:

- **Empty `win_predictions.csv` crash** — a clip with zero detected rallies made `predict.py`'s `pd.DataFrame(rows)` produce a 0-column (not just 0-row) frame, which `.to_csv()`'d to an effectively empty file that `pd.read_csv()` downstream couldn't parse at all (`EmptyDataError`). Fixed by always passing an explicit `columns=` list to `pd.DataFrame()`, so even zero rallies produces a valid, header-only CSV.
- **`idxmax()` on an empty rally set** — see `07_fusion_commentary_pdf.md`'s zero-rally guard; same root cause (a genuinely rally-free clip) as the bug above, caught one stage later.
- **`stitch_segments.py`'s segment-index regex** — see `02_court_presence_and_segmentation.md`; found because a real test upload happened to produce a match folder whose *name* looked like a segment filename.

- **nextTempId collision on a resumed session** — a newly-added rally's temporary ID always started at `-1` on page load; if a *previous* session's saved corrections already used `-1` for a user-added rally, a new addition in a later session would collide with it. Fixed by computing the starting ID from the lowest existing negative ID already present in loaded corrections, not always resetting to `-1`.

Parameters reference

Parameter	Value	Meaning
Poll interval	2000ms	Processing-screen status check frequency
LOW_CONF_THRESHOLD	0.6	<code>cnn_conf</code> below this flags a rally as low-confidence
Gap-to-addable threshold	14s	Timeline gaps wider than this show an "add missed rally" affordance
PYTHONUNBUFFERED	1	Set in the pipeline subprocess's environment for live progress